The Road to Attention

The fix that worked, and the wall it hit

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The disease, recalled

[53] ended on a number: influence fades as V^k, and $0.8^{29} \approx 0.0015$. Multiplying the same matrix into itself, 29 times, starved the gradient before it ever reached the first token.

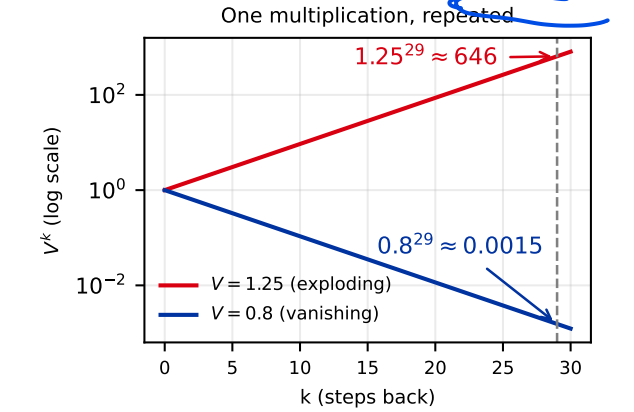
Predict: what arithmetic operation would *not* shrink a signal no matter how many times you repeated it?

The disease, recalled

[53] ended on a number: influence fades as V^k, and $0.8^{29} \approx 0.0015$. Multiplying the same matrix into itself, 29 times, starved the gradient before it ever reached the first token.

Predict: what arithmetic operation would *not* shrink a signal no matter how many times you repeated it?

Addition. Repeated multiplication decays (or explodes); repeated addition just accumulates. Hold that thought - it is the first half of this lecture, and you have already seen it once (ResNet's skip connection, [49]).



Multiplication: the disease. Addition: today's medicine.

Outline

LSTM: the additive fix
Seq2Seq and the wall
Epilogue: recurrence in 2026



▶ Kargin Haghordum, ep. 24

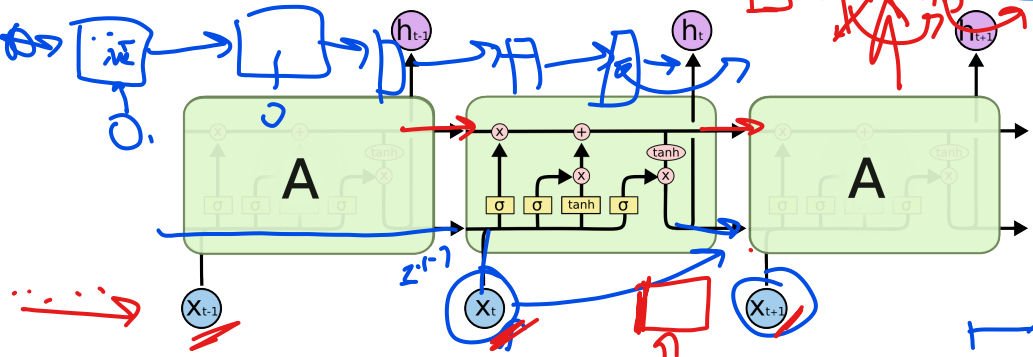
A memory highway

Hochreiter & Schmidhuber's 1997 answer to the disease Hochreiter's own 1991 thesis diagnosed.



The cell state

The **Long Short-Term Memory** (**LSTM**; Hochreiter & Schmidhuber, 1997) adds a second track alongside the hidden state **z**: the **cell state c**, the line along the top. At every step it is only lightly modified - a little is removed, a little is added.

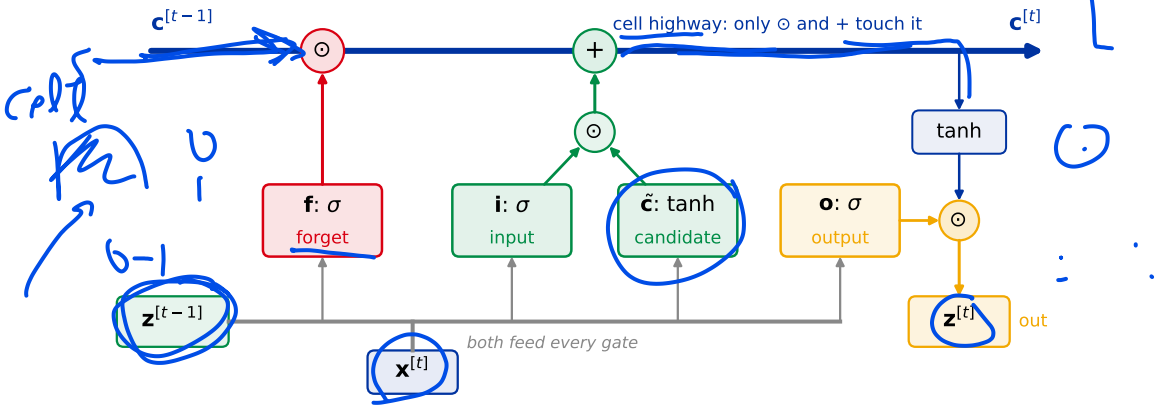


Sorting-line metaphor: the belt carries the box past every station; each worker only adds or removes a little. Nobody rebuilds the box at every station.

Every horizontal line is a full vector; circles are element-wise operations; yellow boxes are learned layers.

Anatomy of one LSTM step

Three gates and one candidate, each computed from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$; each gate is a sigmoid vector in $(0, 1)$ used as an **element-wise mask** ($\odot$).



Forget: drop the old subject when a new one appears. **Input:** **i** says *how much* to write, the candidate $\tilde{\mathbf{c}}$ says *what* (a strong opinion word, say). **Output:** **c** is the cell's private memory; $\mathbf{z} = \mathbf{o} \odot \tanh(\mathbf{c})$ is the part it shows to the next step and the readout - only what is needed right now.

A gate is a soft switch

A gate is [41_1]'s sigmoid neuron, rebranded: one number in $(0, 1)$ per memory slot, multiplied element-wise into the cell.

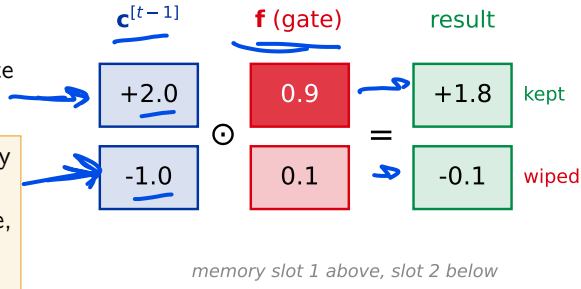
Predict: forget gate $[0.9, 0.1]$ meets cell state $[2.0, -1.0]$ - what survives?

A gate is a soft switch

A gate is [41_1]'s sigmoid neuron, rebranded: one number in $(0, 1)$ per memory slot, multiplied element-wise into the cell.

Predict: forget gate $[0.9, 0.1]$ meets cell state $[2.0, -1.0]$ - what survives?

$[0.9, 0.1] \odot [2.0, -1.0] = [1.8, -0.1]$: memory 1 survives almost untouched, memory 2 is nearly wiped. Near 1 = keep, near 0 = erase, and the network picks the value at every step.



Why a candidate $\tilde{\mathbf{c}}$, and why sigmoid?



$\tilde{\mathbf{c}}$ is **[53]'s RNN step**, the same formula:

$$\tilde{\mathbf{c}}^{[t]} = \tanh\left(\mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{b}\right)$$

The vanilla RNN made this its new state: a full rewrite at every step, and that rewrite is the multiplication that starved the gradient. The LSTM treats it as a **proposal** and adds only a dose of it:

$$\mathbf{c}^{[t]} = \mathbf{f} \odot \mathbf{c}^{[t-1]} + \mathbf{i} \odot \tilde{\mathbf{c}}^{[t]}$$

Two questions, two small layers: **what** to write ($\tilde{\mathbf{c}}$) and **how much** of it ($\mathbf{i}$).

Sigmoid for $\mathbf{f}$, $\mathbf{i}$, $\mathbf{o}$: a gate is a **fraction**, 0 = shut, 1 = open. Multiplying by a number in $(0, 1)$ can only keep or shrink - never flip a sign, never blow up.

tanh for $\tilde{\mathbf{c}}$: content needs a **sign**. "Sweet" pushes a mood slot up, "rotten" pushes it down; a sigmoid could only ever push up. The bound $(-1, 1)$ keeps one step from writing a huge number.

Why not a hard 0/1 switch? Its slope is 0 everywhere except at the jump, so backprop could never tune it. A sigmoid is a switch you can differentiate.

One full LSTM step, in numbers

The same two memory slots, now with every gate. Made-up gate values - a real cell computes each one from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$ with its own small layer: $\mathbf{f} = [0.9, 0.1]$, $\mathbf{i} = [0.2, 0.9]$, $\tilde{\mathbf{c}} = [0.5, 0.8]$, $\mathbf{o} = [0.95, 0.05]$. **(Click to advance.)**

keep $\mathbf{f} \odot \mathbf{c}^{[t-1]} = [0.9,\ 0.1] \odot [2.0,\ -1.0] = [1.8,\ -0.1]$

One full LSTM step, in numbers

The same two memory slots, now with every gate. Made-up gate values - a real cell computes each one from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$ with its own small layer: $\mathbf{f} = [0.9, 0.1]$, $\mathbf{i} = [0.2, 0.9]$, $\tilde{\mathbf{c}} = [0.5, 0.8]$, $\mathbf{o} = [0.95, 0.05]$. **(Click to advance.)**

keep $\mathbf{f} \odot \mathbf{c}^{[t-1]} = [0.9,\ 0.1] \odot [2.0,\ -1.0] = [1.8,\ -0.1]$

write $\mathbf{i} \odot \tilde{\mathbf{c}} = [0.2,\ 0.9] \odot [0.5,\ 0.8] = [0.10,\ 0.72]$

One full LSTM step, in numbers

The same two memory slots, now with every gate. Made-up gate values - a real cell computes each one from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$ with its own small layer: $\mathbf{f} = [0.9, 0.1]$, $\mathbf{i} = [0.2, 0.9]$, $\tilde{\mathbf{c}} = [0.5, 0.8]$, $\mathbf{o} = [0.95, 0.05]$. **(Click to advance.)**

keep $\mathbf{f} \odot \mathbf{c}^{[t-1]} = [0.9,\ 0.1] \odot [2.0,\ -1.0] = [1.8,\ -0.1]$

write $\mathbf{i} \odot \tilde{\mathbf{c}} = [0.2,\ 0.9] \odot [0.5,\ 0.8] = [0.10,\ 0.72]$

new cell $\mathbf{c}^{[t]} = \text{keep} + \text{write} = [1.90,\ 0.62]$

One full LSTM step, in numbers

The same two memory slots, now with every gate. Made-up gate values - a real cell computes each one from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$ with its own small layer: $\mathbf{f} = [0.9, 0.1]$, $\mathbf{i} = [0.2, 0.9]$, $\tilde{\mathbf{c}} = [0.5, 0.8]$, $\mathbf{o} = [0.95, 0.05]$. **(Click to advance.)**

keep $\mathbf{f} \odot \mathbf{c}^{[t-1]} = [0.9,\ 0.1] \odot [2.0,\ -1.0] = [1.8,\ -0.1]$

write $\mathbf{i} \odot \tilde{\mathbf{c}} = [0.2,\ 0.9] \odot [0.5,\ 0.8] = [0.10,\ 0.72]$

new cell $\mathbf{c}^{[t]} = \text{keep} + \text{write} = [1.90,\ 0.62]$

show $\mathbf{z}^{[t]} = \mathbf{o} \odot \tanh(\mathbf{c}^{[t]}) = [0.95,\ 0.05] \odot [0.96,\ 0.55] = [0.91,\ 0.03]$

One full LSTM step, in numbers

The same two memory slots, now with every gate. Made-up gate values - a real cell computes each one from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$ with its own small layer: $\mathbf{f} = [0.9, 0.1]$, $\mathbf{i} = [0.2, 0.9]$, $\tilde{\mathbf{c}} = [0.5, 0.8]$, $\mathbf{o} = [0.95, 0.05]$. **(Click to advance.)**

keep $\mathbf{f} \odot \mathbf{c}^{[t-1]} = [0.9,\ 0.1] \odot [2.0,\ -1.0] = [1.8,\ -0.1]$

write $\mathbf{i} \odot \tilde{\mathbf{c}} = [0.2,\ 0.9] \odot [0.5,\ 0.8] = [0.10,\ 0.72]$

new cell $\mathbf{c}^{[t]} = \text{keep} + \text{write} = [1.90,\ 0.62]$

show $\mathbf{z}^{[t]} = \mathbf{o} \odot \tanh(\mathbf{c}^{[t]}) = [0.95,\ 0.05] \odot [0.96,\ 0.55] = [0.91,\ 0.03]$

Slot 1 keeps its memory and shows it. **Slot 2** drops the old -1, writes 0.72 instead - and keeps it private: it holds 0.62 but shows only 0.03. **i** decides how much to write, $\tilde{\mathbf{c}}$ what, **o** who gets to see it.



What is learned, what is computed

A toy cell: **one** memory slot that tracks the review's mood, and each word comes in as two numbers, $\mathbf{x} = [\text{good}, \text{bad}]$ (no $\mathbf{V}$, no output gate). The weights are **made up** to be readable; training finds messier ones that do the same job.

Learned by backprop, then fixed: 9 numbers.

$\tilde{c} = \tanh(\mathbf{W}^\top \mathbf{x} + b)$ — which way?
$\mathbf{W} = [+2, -2]$, $b = 0$

$i = \sigma(\mathbf{W}_i^\top \mathbf{x} + b_i)$ — opinion word?
$\mathbf{W}_i = [+5, +5]$, $b_i = -3$

$f = \sigma(\mathbf{W}_f^\top \mathbf{x} + b_f)$ — keep 0.95
$\mathbf{W}_f = [0, 0]$, $b_f = +3$

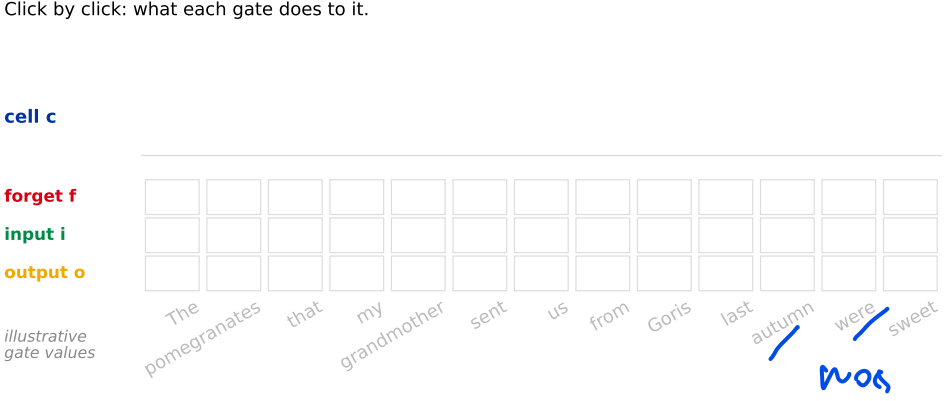
Computed fresh at every word, from those 9 numbers. Rule: $c \leftarrow 0.95\, c + i\, \tilde{c}$, starting from $c = 0$.

word	$\mathbf{x}$	$\tilde{c}$	i	new c
The … arrived	$[0, 0]$	0	0.05	0
fresh	$[1, 0]$	$+0.96$	0.88	0.85
and	$[0, 0]$	0	0.05	0.81
sweet	$[1, 0]$	$+0.96$	0.88	**1.62**
rotten, not sweet	$[0, 1]$	-0.96	0.88	-0.08

Same input, two readings: "sweet" and "rotten" open the gate equally and write opposite signs. A real cell learns the same kind of weights, as matrices - 1,152 numbers for a 16-slot cell - and nobody picks them by hand.

Watching one memory slot

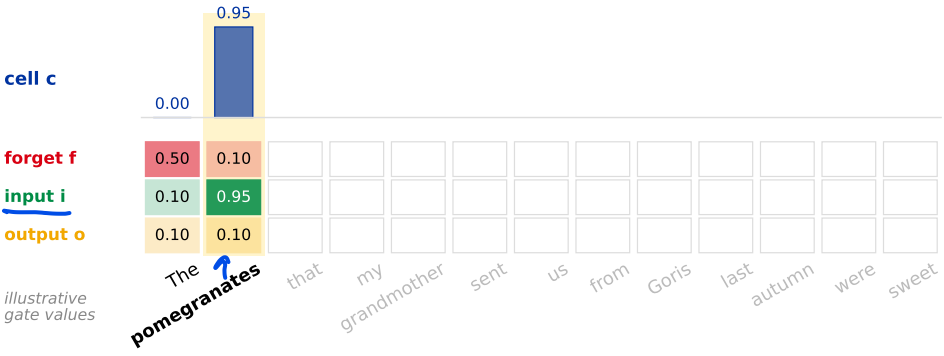
The sentence from [53]'s fixed-window frame, read by an LSTM. One slot of its cell has learned one job: remember whether the subject is plural. The gate values are **made up** to show the mechanism - a trained cell is messier (measured results in a few frames). **(Click to advance.)**



Watching one memory slot

The sentence from [53]'s fixed-window frame, read by an LSTM. One slot of its cell has learned one job: remember whether the subject is plural. The gate values are **made up** to show the mechanism - a trained cell is messier (measured results in a few frames). **(Click to advance.)**

"pomegranates": the input gate opens (0.95) and writes +1, "plural",
into the cell: c = 0.95.

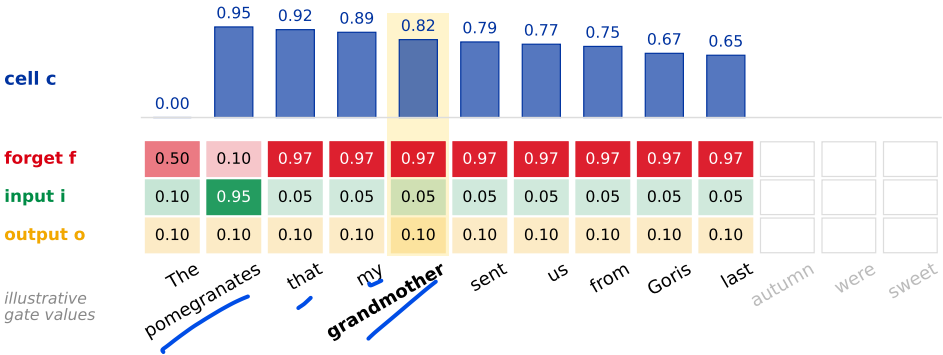


Watching one memory slot

The sentence from [53]'s fixed-window frame, read by an LSTM. One slot of its cell has learned one job: remember whether the subject is plural. The gate values are **made up** to show the mechanism - a trained cell is messier (measured results in a few frames). **(Click to advance.)**

"grandmother" is singular, but the input gate stays shut (0.05): c only goes 0.89 -> 0.82.

The forget gate holds the memory at 0.97 per word.

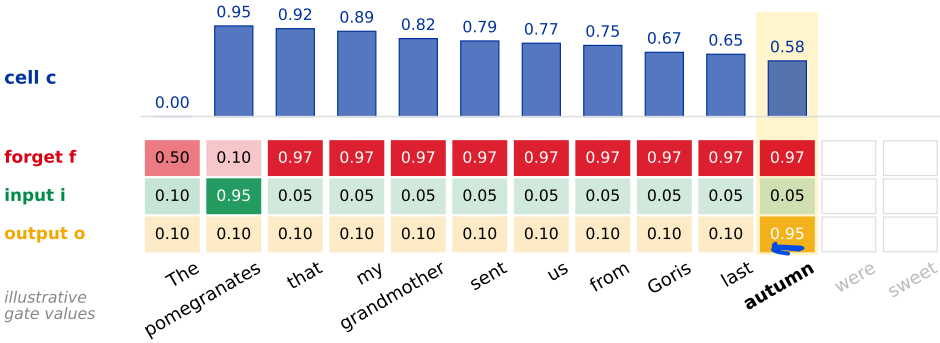


Watching one memory slot

The sentence from [53]'s fixed-window frame, read by an LSTM. One slot of its cell has learned one job: remember whether the subject is plural. The gate values are **made up** to show the mechanism - a trained cell is messier (measured results in a few frames). **(Click to advance.)**

"autumn": the next word is the verb, so the output gate opens (0.95):

z = 0.95 x tanh(0.58) = 0.50 reaches the readout, which picks "were", not "was".

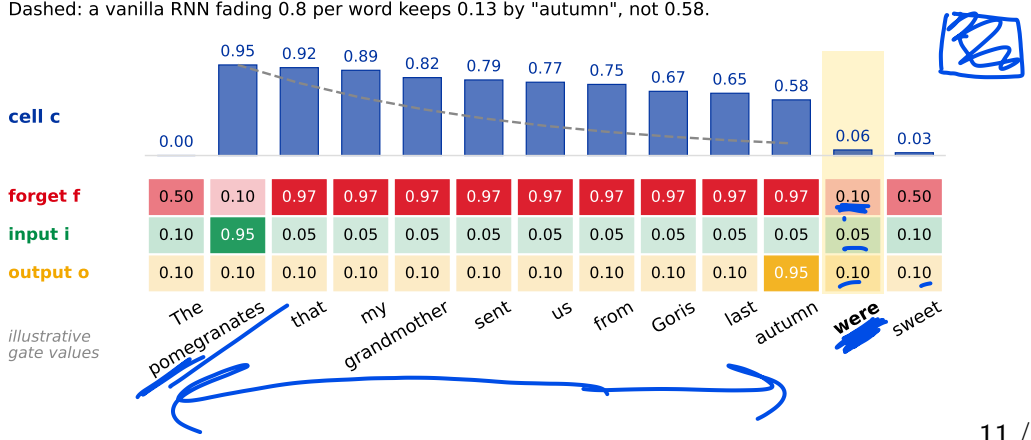


Watching one memory slot

The sentence from [53]'s fixed-window frame, read by an LSTM. One slot of its cell has learned one job: remember whether the subject is plural. The gate values are **made up** to show the mechanism - a trained cell is messier (measured results in a few frames). **(Click to advance.)**

"were": the subject is used up - the forget gate drops to 0.1 and wipes the slot (c = 0.06).

Dashed: a vanilla RNN fading 0.8 per word keeps 0.13 by "autumn", not 0.58.



For the record: the four equations

Reference, not exam material - shown once.

Forget gate: $\mathbf{f}^{[t]} = \sigma\big(\mathbf{W}_f^\top \mathbf{x}^{[t]} + \mathbf{V}_f^\top \mathbf{z}^{[t-1]} + \mathbf{b}_f\big)$

Input gate: $\mathbf{i}^{[t]} = \sigma\big(\mathbf{W}_i^\top \mathbf{x}^{[t]} + \mathbf{V}_i^\top \mathbf{z}^{[t-1]} + \mathbf{b}_i\big)$

$\tilde{\mathbf{c}}^{[t]} = \tanh\big(\mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{b}\big)$

Cell update: $\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \mathbf{i}^{[t]} \odot \tilde{\mathbf{c}}^{[t]}$

Output gate: $\mathbf{o}^{[t]} = \sigma\big(\mathbf{W}_o^\top \mathbf{x}^{[t]} + \mathbf{V}_o^\top \mathbf{z}^{[t-1]} + \mathbf{b}_o\big)$

$\mathbf{z}^{[t]} = \mathbf{o}^{[t]} \odot \tanh\big(\mathbf{c}^{[t]}\big)$

The punchline: $\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \ldots$ is **additive** - gradients flow through the cell state with minimal decay. ResNet solved *depth* with addition in 2015; LSTM solved *time* with addition in 1997 - same medicine, 18 years earlier.

Why addition keeps the gradient alive

Follow the gradient backwards along the highway only (ignore the paths through the gates). One line of calculus:

$$\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \mathbf{i}^{[t]} \odot \tilde{\mathbf{c}}^{[t]} \quad \Longrightarrow \quad \frac{\partial \mathbf{c}^{[t]}}{\partial \mathbf{c}^{[t-1]}} = \operatorname{diag}\left(\mathbf{f}^{[t]}\right)$$

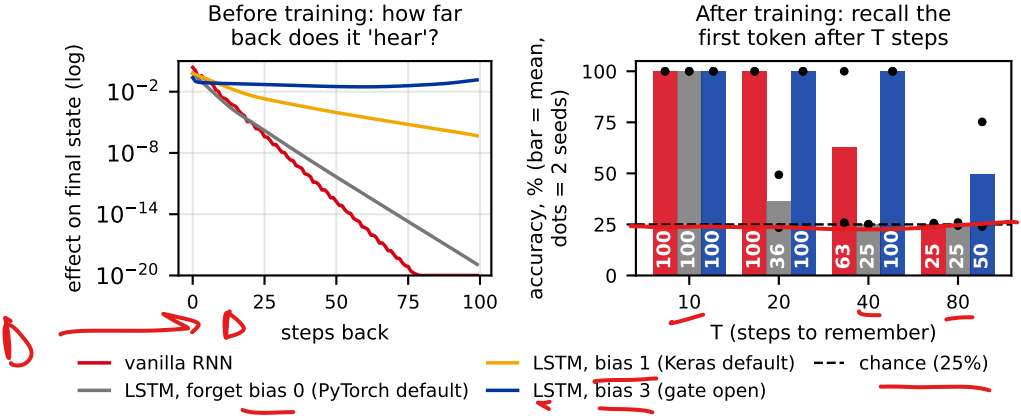
The other paths back run through the gates and their weight matrices, and those do fade like an RNN's. The highway does not have to - and one path that survives is enough to carry the signal.

Vanilla RNN ([53]): each step multiplies by $\mathbf{D}^{[t-1]}\mathbf{V}^{\top}$ - the *same* matrix every step, whether or not the memory matters. Its size decides the fate of everything.

LSTM highway: each step multiplies by $\mathbf{f}^{[t]}$, which the network *recomputes* every step, per slot: ≈ 1 to hold, ≈ 0 to wipe. Close to 1 matters: $0.97^{100} \approx 0.05$, $0.99^{100} \approx 0.37$ (vanilla: $0.8^{100} \approx 2 \cdot 10^{-10}$).

History: the 1997 LSTM had *no* forget gate - a "constant error carousel" with factor exactly 1. On long, unsegmented input its cell state kept growing until the network broke down; the forget gate (Gers, Schmidhuber & Cummins, 2000) lets it reset.

Does it work? Measured



Left, before training: an input 40 steps back barely moves a vanilla RNN or a half-shut LSTM (bias 0); bias 1 decays slower; open gates (bias 3) still count at 100. **Right**, same budget: the open-gate LSTM solves $T \leq 40$ in 100 steps; at $T = 40$ the RNN solves one seed of two, at 80 neither, and even the open-gate LSTM reaches only 75% on one seed. Addition makes long memory *learnable*, not free.

The price, and one gotcha

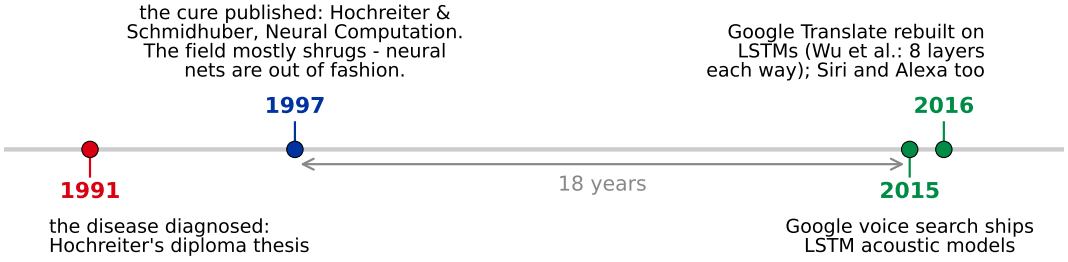
- **Price:** 4 weight sets instead of 1. Our d_x=1, d_h=16 vanilla cell: $16\,(\mathbf{W}) + 256\,(\mathbf{V}) + 16\,(\mathbf{b}) = 288$ params. LSTM: $4 \times 288 = \mathbf{1{,}152}$. (PyTorch's `nn.LSTM` keeps two bias vectors, so it reports 1,216.)
- **Gated Recurrent Unit** (**GRU**; Cho et al., 2014): a streamlined 2-gate cousin (3 weight sets), similar results - the lighter default.
- **Gotcha:** the measured frame's "gate starts open" (bias 3) is a one-line choice. The gate starts at $\sigma(\text{bias})$: $\sigma(0) = 0.5$ halves the memory every step, $\sigma(3) \approx 0.95$ keeps it. Keras adds $+1$ to the forget-gate bias by default; **PyTorch does not**:

```
lstm = nn.LSTM(d_in, hidden)
# PyTorch stacks the gates: input, forget, cell, output
lstm.bias_ih_l0.data[hidden:2 * hidden].fill_(1.0)  # forget gate
```



Same lesson as [45]'s initialization frames: where training *starts* can decide whether it gets anywhere at all.

Published 1997, everywhere by 2016



18 years from publication to running in everyone's pocket - and one of the most-cited AI papers ever (80,000+ citations). When a 2010s product said "powered by neural networks", the network was very often an LSTM.

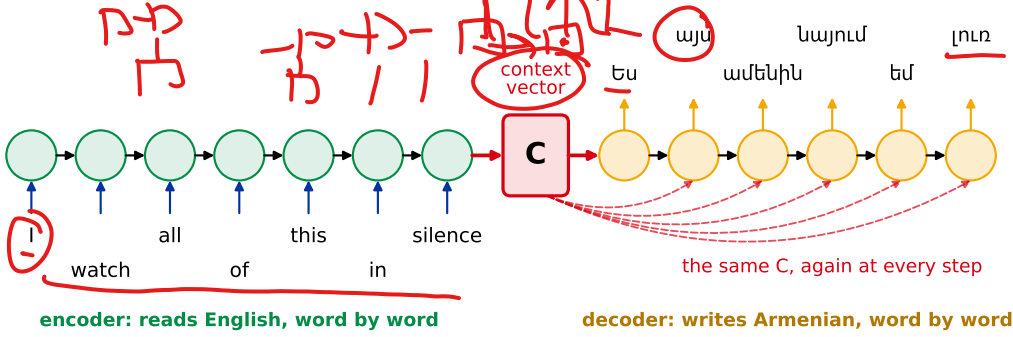
Fine print, for next chapter: 2016's Google Translate already carried an *attention* link between its encoder and decoder - the idea beyond LSTM was creeping in even at LSTM's moment of triumph.

Two networks, one vector between them

Translation needs a different-length output. Here is the classic fix.

Encoder-decoder

Translation needs different input and output lengths. Fix: an **encoder** LSTM reads the source; its last state is ONE **context vector** C. A **decoder** LSTM starts from C (and, in Cho et al.'s version, gets it again at every step, dashed) and writes the target one word at a time - each written word fed back in as its next input.



Running example: "I watch all of this in silence, and the connoisseurs speak inside me." goes in, and the decoder writes the chapter's Armenian line from [53] (first half shown). The 2014 recipe that made neural machine translation real: Sutskever, Vinyals & Le (2014); Cho et al. (2014).

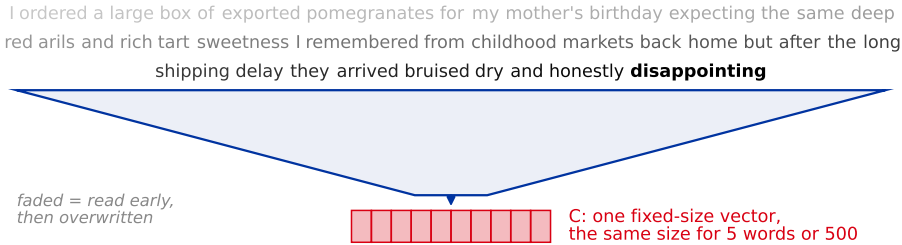
The bottleneck

Predict: the 42-word complaint review ([53]'s shoehorn-hacks frame - it lost its verdict to truncation already) squeezed into a single 300-dim context vector. What goes wrong?



The bottleneck

Predict: the 42-word complaint review ([53]'s shoehorn-hacks frame - it lost its verdict to truncation already) squeezed into a single 300-dim context vector. What goes wrong?

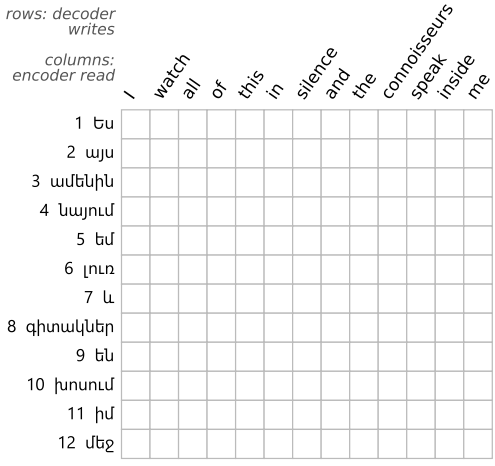


Lost: early words get overwritten as the state updates - "expecting", "childhood markets" fade first.

Fixed capacity: 5 words or 500, the same-size C must hold them all.

And the LSTM does NOT save us: the cell state is still one fixed-size vector. The fix for *fading* memory did not fix *compressed* memory.

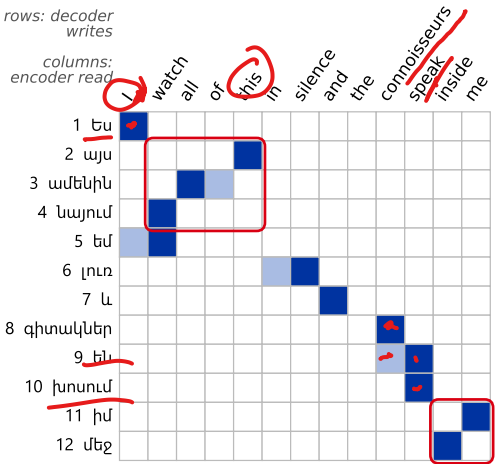
The real problem: every step gets the same C



So just make C bigger? First look at what the decoder actually *needs*.

Predict: the decoder writes the Armenian line one word per row. For each row, which English words does it need?

The real problem: every step gets the same C





So just make C bigger? First look at what the decoder actually *needs*.

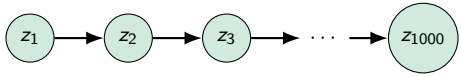
Predict: the decoder writes the Armenian line one word per row. For each row, which English words does it need?

Each row needs **one or two** words - a **different** few every time, and not even in order: rows 2-4 run backwards ("this all watch"), and so do rows 11-12 ("my inside").

Seq2seq hands **every** step the **same** C. The deeper problem is not that C is small: each step needs a **different part** of the input, and nothing lets the decoder go and get it.

The second wall: one step at a time

Even a perfect LSTM has one more problem, and it is about **speed**, not memory: $\mathbf{z}^{[t]}$ needs $\mathbf{z}^{[t-1]}$, which needs $\mathbf{z}^{[t-2]}$ ... Step t cannot start until step $t-1$ has finished.



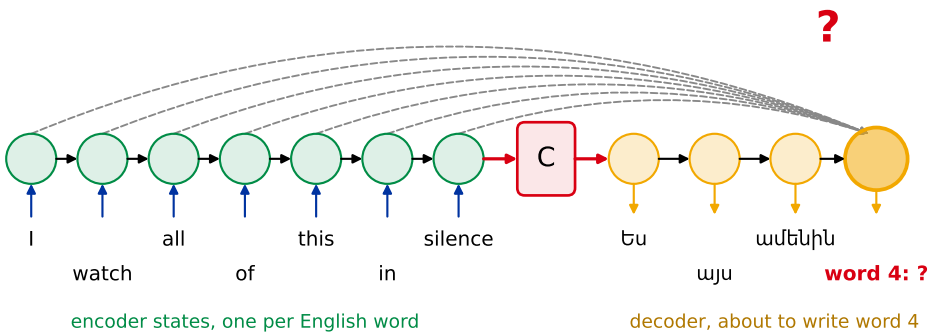
1,000 tokens = 1,000 steps
in a queue, one after another

- The GPUs from [42] want thousands of *independent* multiply-adds at once. A recurrence hands them one small step at a time - most of the chip waits.
- The authors of the paper that ended the RNN era put it plainly: "This inherently sequential nature precludes parallelization within training examples" (Vaswani et al., 2017).

Two walls, then: the **bottleneck** (one vector, the same for every step, must hold everything) and the **queue** (one step at a time). The next chapter takes them down with two moves: let the decoder look back at every encoder state (the bottleneck), and drop the recurrence so all words are read at once (the queue).

The cliffhanger

What if the decoder could look back at *all* the encoder states - and learn *where* to look, a different place for every word it writes?



That question has a name. It was answered between 2014 and 2017, and it ends this course's pre-history of sequence models. Next chapter: attention and transformers.

Recurrence, retirement, and a comeback



2017's transformer retired RNNs from mainstream NLP. But recurrence is back: several 2025-26 LLMs run most of their layers as a fixed-size recurrent state with a learned *decay* gate - today's forget gate. Why it came back makes sense only once you know what it replaced: the LLM chapter.

Recap

- Length varies, memory fades ([53]) $\rightarrow$ LSTM fixes fading with an **additive** cell state - same medicine as ResNet, 18 years earlier - at $4\times$ the parameter cost. Its gates decide, per memory slot, what to write, keep and release.
- **Seq2seq** squeezes the whole source sentence into one vector C and hands **every** output step that **same** C - while each step needs a **different** part of the input. And any recurrence runs **one step at a time**.

Next: the LLM chapter. It starts where text enters the model - what exactly is a token? - and then answers today's question: let the decoder look back at everything and take a different part for each word it writes. That is attention (LLM-3); dropping the recurrence altogether also removes the one-step-at-a-time queue.